부록 · 리눅스 명령어

리눅스 공통 자주 사용하는 명령어 170선

Ubuntu · Debian · Rocky Linux · Fedora · 일반 Linux 서버 관리용

리눅스에서 공통적으로 자주 사용하는 명령어를 파일·디렉터리, 검색, 압축, 시스템, 프로세스, 서비스, 네트워크, 사용자·권한, 패키지, 서버 관리 및 Spring Boot·MariaDB 실무 순으로 정리함.

이 문서에서 고친 곳

- usermod -aG sudo 는 Ubuntu 기준입니다. 이 자료의 서버(Rocky Linux)는 관리자 그룹이 wheel 이라 고쳤습니다.
- netstat 은 Rocky Linux 기본 설치에 없어 net-tools 설치를 덧붙였습니다. 같은 일은 ss -tulpn 으로 바로 됩니다.

1. 파일·디렉터리 기본 명령어

번호	명령어	기능	사용 예
1	pwd	현재 위치 확인	pwd
2	ls	파일 목록	ls
3	ls -l	상세 목록	ls -l
4	ls -a	숨김 파일 포함	ls -a
5	ls -lh	파일 크기를 읽기 쉽게 표시	ls -lh
6	cd	디렉터리 이동	cd /home
7	cd ..	상위 디렉터리 이동	cd ..
8	cd ~	홈 디렉터리 이동	cd ~
9	mkdir	디렉터리 생성	mkdir project
10	mkdir -p	하위 경로까지 생성	mkdir -p app/logs
11	rmdir	빈 디렉터리 삭제	rmdir test
12	touch	빈 파일 생성/시간 갱신	touch test.txt
13	cp	파일 복사	cp a.txt b.txt
14	cp -r	디렉터리 복사	cp -r app backup
15	mv	이동/이름 변경	mv old.txt new.txt
16	rm	파일 삭제	rm test.txt
17	rm -r	디렉터리 삭제	rm -r test
18	rm -rf	강제 재귀 삭제	rm -rf test
19	file	파일 종류 확인	file app.jar
20	stat	파일 상세정보	stat test.txt

2. 파일 내용 확인·편집

번호	명령어	기능	사용 예
21	cat	파일 전체 출력	cat application.yml
22	less	긴 파일 페이지 단위 확인	less server.log
23	more	파일 페이지 출력	more test.txt
24	head	앞부분 확인	head server.log
25	head -n	앞 N줄 확인	head -n 20 server.log
26	tail	마지막 부분 확인	tail server.log
27	tail -f	실시간 로그 확인	tail -f app.log
28	nano	간단한 텍스트 편집	nano application.yml
29	vi	vi 편집기	vi test.txt
30	vim	Vim 편집기	vim test.txt

3. 검색·문자열 처리

번호	명령어	기능	사용 예
31	find	파일 검색	find /home -name "*.jar"
32	locate	파일명 DB 검색	locate application.yml
33	which	실행파일 위치	which java
34	whereis	프로그램 관련 위치	whereis java
35	grep	문자열 검색	grep "ERROR" app.log
36	grep -i	대소문자 무시	grep -i "error" app.log
37	grep -r	하위 디렉터리까지 검색	grep -r "3306" /etc
38	sort	정렬	sort names.txt
39	uniq	중복 제거	sort a.txt | uniq
40	wc	줄/단어/문자 수	wc -l app.log
41	cut	특정 열 추출	cut -d: -f1 /etc/passwd
42	awk	열/텍스트 처리	awk '{print $1}' file.txt
43	sed	문자열 치환/편집	sed 's/8080/8081/g' a.txt
44	tr	문자 변환	tr 'a-z' 'A-Z'
45	xargs	입력을 명령 인자로 전달	find . -name "*.log" | xargs ls -lh

4. 압축·백업

번호	명령어	기능	사용 예
46	tar	파일 묶기	tar -cvf backup.tar app/

번호	명령어	기능	사용 예
47	tar -xvf	tar 풀기	tar -xvf backup.tar
48	tar -czvf	gzip tar 압축	tar -czvf app.tar.gz app/
49	tar -xzvf	tar.gz 해제	tar -xzvf app.tar.gz
50	gzip	gzip 압축	gzip server.log
51	gunzip	gzip 해제	gunzip server.log.gz
52	zip	ZIP 압축	zip -r app.zip app/
53	unzip	ZIP 해제	unzip app.zip
54	rsync	파일 동기화/백업	rsync -av source/ backup/

5. 디스크·메모리·시스템 확인

번호	명령어	기능	사용 예
55	df	디스크 사용량	df -h
56	du	파일/폴더 용량	du -sh /home
57	free	메모리 사용량	free -h
58	top	실시간 프로세스/자원	top
59	htop	편리한 프로세스 모니터	htop
60	uptime	가동시간/부하	uptime
61	uname	시스템 정보	uname -a
62	hostname	호스트명 확인	hostname
63	hostnamectl	호스트/OS 정보	hostnamectl
64	lsblk	디스크·파티션 확인	lsblk
65	blkid	UUID/파일시스템 확인	sudo blkid
66	mount	파일시스템 마운트	mount
67	umount	마운트 해제	sudo umount /mnt/data
68	lscpu	CPU 정보	lscpu
69	lsusb	USB 장치 확인	lsusb
70	lspci	PCI 장치 확인	lspci

6. 프로세스 관리

번호	명령어	기능	사용 예
71	ps	프로세스 확인	ps aux
72	pgrep	이름으로 PID 검색	pgrep java
73	pidof	프로그램 PID 확인	pidof java

번호	명령어	기능	사용 예
74	kill	PID 프로세스 종료	kill 1234
75	kill -9	프로세스 강제 종료	kill -9 1234
76	pkill	이름으로 종료	pkill java
77	jobs	백그라운드 작업 확인	jobs
78	bg	작업을 백그라운드로	bg %1
79	fg	작업을 포그라운드로	fg %1
80	nohup	로그아웃 후에도 실행	nohup java -jar app.jar &

7. 서비스·로그 관리

번호	명령어	기능	사용 예
81	systemctl status	서비스 상태	systemctl status mariadb
82	systemctl start	서비스 시작	sudo systemctl start mariadb
83	systemctl stop	서비스 중지	sudo systemctl stop mariadb
84	systemctl restart	서비스 재시작	sudo systemctl restart nginx
85	systemctl enable	부팅 시 자동 시작	sudo systemctl enable nginx
86	systemctl disable	자동 시작 해제	sudo systemctl disable nginx
87	systemctl list-units	서비스 목록	systemctl list-units --type=service
88	journalctl	systemd 로그	journalctl
89	journalctl -u	특정 서비스 로그	journalctl -u mariadb
90	journalctl -f	실시간 시스템 로그	journalctl -f

8. 네트워크·포트·인터넷

번호	명령어	기능	사용 예
91	ip addr	IP 주소 확인	ip addr
92	ip link	NIC 상태	ip link
93	ip route	라우팅 확인	ip route
94	ping	통신 테스트	ping 8.8.8.8
95	ss	포트/소켓 확인	ss -tulpn
96	netstat	네트워크 상태	netstat -tulpn (Rocky 는 먼저 sudo dnf install -y net-tools)
97	curl	HTTP/API 요청	curl http://localhost:8080
98	wget	파일 다운로드	wget URL
99	traceroute	네트워크 경로 추적	traceroute google.com
100	nslookup	DNS 조회	nslookup google.com

번호	명령어	기능	사용 예
101	dig	상세 DNS 조회	dig google.com
102	nc	포트 연결 테스트	nc -zv SERVER_IP 3306
103	ssh	원격 SSH 접속	ssh user@SERVER_IP
104	scp	SSH 파일 복사	scp app.jar user@server:/home/user/
105	sftp	SSH 파일 전송	sftp user@server

9. 사용자·권한 관리

번호	명령어	기능	사용 예
106	whoami	현재 사용자	whoami
107	who	로그인 사용자	who
108	id	UID/GID 확인	id devuser
109	sudo	관리자 권한 실행	sudo systemctl restart nginx
110	su	사용자 전환	su - root
111	passwd	비밀번호 변경	passwd
112	useradd	사용자 생성	sudo useradd test
113	userdel	사용자 삭제	sudo userdel test
114	usermod	사용자 설정 변경	sudo usermod -aG wheel test (Ubuntu 는 sudo 그룹)
115	groupadd	그룹 생성	sudo groupadd developer
116	groups	소속 그룹 확인	groups
117	chmod	권한 변경	chmod 755 app.sh
118	chown	소유자 변경	sudo chown user:user app.jar
119	chgrp	그룹 변경	chgrp developer app.jar
120	umask	기본 권한 마스크	umask

10. 패키지 설치·관리

번호	명령어	기능	사용 예
121	apt update	Ubuntu/Debian 패키지 목록 갱신	sudo apt update
122	apt upgrade	Ubuntu/Debian 패키지 업그레이드	sudo apt upgrade
123	apt install	Ubuntu/Debian 패키지 설치	sudo apt install nginx
124	apt remove	Ubuntu/Debian 패키지 제거	sudo apt remove nginx
125	dpkg	Debian 패키지 조회	dpkg -l
126	dnf install	Rocky/Fedora 패키지 설치	sudo dnf install nginx
127	dnf update	Rocky/Fedora 업데이트	sudo dnf update

번호	명령어	기능	사용 예
128	dnf remove	Rocky/Fedora 패키지 제거	sudo dnf remove nginx
129	rpm	RPM 패키지 조회	rpm -qa
130	yum	RHEL 계열 패키지 관리	sudo yum install nginx

11. 서버 관리에서 자주 사용하는 명령어

번호	명령어	기능	사용 예
131	history	명령 기록 확인	history
132	clear	화면 지우기	clear
133	date	날짜/시간 확인	date
134	timedatectl	시간대/시간 설정 확인	timedatectl
135	env	환경변수 확인	env
136	echo	문자열/변수 출력	echo $PATH
137	export	환경변수 설정	export JAVA_HOME=/usr/lib/jvm/...
138	alias	명령 별칭	alias ll='ls -alF'
139	source	설정 다시 읽기	source ~/.bashrc
140	crontab	예약 작업	crontab -e
141	reboot	시스템 재부팅	sudo reboot
142	shutdown	시스템 종료	sudo shutdown -h now
143	poweroff	전원 종료	sudo poweroff
144	dmesg	커널 메시지 확인	dmesg | tail
145	last	로그인 기록	last
146	w	로그인 사용자/작업	w
147	watch	명령 반복 실행	watch -n 2 free -h
148	tee	화면 출력과 파일 저장	echo test | tee test.txt
149	ln	링크 생성	ln -s /data/app app
150	realpath	절대경로 확인	realpath app.jar

12. Spring Boot·MariaDB 실무 명령어

번호	명령어	기능	사용 예
151	java -version	Java 버전 확인	java -version
152	which java	Java 실행파일 위치	which java
153	java -jar	Spring Boot JAR 실행	java -jar app.jar
154	nohup java -jar	Spring Boot 백그라운드 실행	nohup java -jar app.jar > app.log 2>&1 &

번호	명령어	기능	사용 예
155	ps + grep	Java 프로세스 검색	ps aux | grep java
156	pgrep -af	Java PID와 명령행 확인	pgrep -af java
157	ss + grep 8080	8080 포트 확인	ss -tulpn | grep 8080
158	kill PID	프로세스 정상 종료	kill PID
159	curl localhost	Spring Boot API 테스트	curl http://localhost:8080
160	mysql	MariaDB 접속	mysql -u root -p
161	systemctl status mariadb	MariaDB 상태 확인	systemctl status mariadb
162	systemctl restart mariadb	MariaDB 재시작	sudo systemctl restart mariadb
163	ss + grep 3306	MariaDB 3306 포트 확인	ss -tulpn | grep 3306
164	tail -f app.log	Spring 로그 실시간 확인	tail -f app.log
165	grep error	로그 에러 검색	grep -i error app.log
166	find JAR	JAR 파일 검색	find / -name "*.jar" 2>/dev/null
167	df -h	디스크 여유 공간 확인	df -h
168	du + sort	큰 디렉터리 확인	du -sh * | sort -h
169	free -h	메모리 확인	free -h
170	top	CPU/프로세스 확인	top

실무 학습 포인트

- 처음에는 모든 명령어를 암기하기보다 ls, cd, pwd, cp, mv, rm, mkdir, cat, tail -f, grep, find를 우선 익히는 것이 좋음.
- 서버 관리에서는 df -h, du -sh, free -h, ps, kill, systemctl, journalctl, ip addr, ss -tulpn, ping, curl, ssh, scp가 특히 중요함.
- Ubuntu/Debian은 주로 apt를 사용하고 Rocky Linux/Fedora 계열은 dnf를 사용하므로 패키지 관리 명령은 배포판별 차이를 구분해야 함.
- rm -rf, kill -9, reboot, shutdown, chmod, chown 등의 명령은 시스템에 큰 영향을 줄 수 있으므로 실행 전에 대상과 경로를 반드시 확인해야 함.

부록 · 리눅스 명령어

Linux sed · awk 유용한 실전 예제 30개

JSP · Spring Boot · properties · conf · 로그 · Linux 서버 관리 실습

1. sed와 awk의 핵심 차이

sed는 스트림 편집기로 문자열 치환, 특정 줄 삭제, 삽입, 범위 출력 등에 강함. awk는 조건에 맞는 행 선택, 필드 분리, 줄 번호 표시, 계산 및 통계 처리에 강함.

기억법:

```bash
sed = 편집(치환·삭제·삽입), awk = 분석(조건·범위·필드·계산).
```

구분	주요 역할	대표 문법	실무 용도
sed	텍스트 편집	s/OLD/NEW/g	설정 변경, 줄 삭제, 삽입
awk	텍스트 분석	조건 {동작}	로그 분석, 필드 추출, 통계

2. sed 실전 예제 15개

1. 문자열 치환

파일을 수정하지 않고 8080을 8081로 바꾼 결과를 화면에 표시함.

```bash
sed 's/8080/8081/g' application.properties
```

2. 실제 파일 수정

-i 옵션으로 파일 자체를 수정함.

```bash
sed -i 's/8080/8081/g' application.properties
```

3. 백업 후 수정

원본을 .bak 파일로 보관하고 수정함.

```bash
sed -i.bak 's/8080/8081/g' application.properties
```

4. 특정 줄 삭제

10번째 줄을 삭제함.

```bash
sed -i.bak '10d' member.jsp
```

5. 범위 삭제

10~20번째 줄을 선택 삭제함.

```bash
sed -i.bak '10,20d' member.jsp
```

6. ERROR 줄 제거

ERROR가 포함된 줄을 제외하여 출력함.

```bash
sed '/ERROR/d' app.log
```

7. 빈 줄 삭제

공백만 있거나 완전히 빈 줄을 제거함.

```bash
sed '/^[[:space:]]*$/d' file.txt
```

8. 주석 줄 제거

#으로 시작하는 설정 주석을 제외함.

```bash
sed '/^[[:space:]]*#/d' application.properties
```

9. 특정 줄만 출력

10번째 줄만 출력함.

```bash
sed -n '10p' member.jsp
```

10. 특정 범위 출력

10~30번째 줄만 편집기 없이 확인함.

```bash
sed -n '10,30p' member.jsp
```

11. 특정 설정 출력

datasource 관련 줄만 출력함.

```bash
sed -n '/spring.datasource/p' application.properties
```

12. 설정 뒤에 내용 삽입

server.port 다음 줄에 server.address를 추가한 결과를 보여줌.

```bash
sed '/server.port/a server.address=0.0.0.0' application.properties
```

13. 설정 앞에 주석 삽입

server.port 앞에 설명 주석을 삽입함.

```bash
sed '/server.port/i # Server Setting' application.properties
```

14. 여러 공백을 하나로 정리

연속된 공백을 하나의 공백으로 정리함.

```bash
sed 's/[[:space:]][[:space:]]*/ /g' file.txt
```

15. IP 주소 일괄 변경

MariaDB나 서버 IP 주소 변경 등에 활용함.

```bash
sed -i.bak 's/192\.168\.0\.10/SERVER_IP/g' application.properties
```

3. sed 추가 실무 응용

기존 server.port 값이 무엇이든 8081로 변경:

```bash
sed -i.bak 's/^server\.port=.*/server.port=8081/' application.properties
```

JSP의 회원가입 문구를 회원등록으로 변경:

```bash
sed -i.bak 's/회원가입/회원등록/g' member.jsp
```

주석과 빈 줄을 동시에 제외:

```bash
sed -e '/^[[:space:]]*#/d' -e '/^[[:space:]]*$/d' application.properties
```

HTML/JSP 태그를 제거하고 텍스트만 추출:

```bash
echo '<h1>Hello Linux</h1>' | sed 's/<[^>]*>//g'
```

여러 properties 파일의 localhost를 일괄 변경:

```bash
find . -name "*.properties" -exec sed -i.bak 's/localhost/SERVER_IP/g' {} +
```

4. awk 실전 예제 15개

16. 전체 내용 출력

파일의 모든 줄을 출력함.

```bash
awk '{print}' file.txt
```

17. 줄 번호와 내용 출력

NR을 이용해 줄 번호를 붙임.

```bash
awk '{print NR ": " $0}' file.txt
```

18. ERROR 검색

ERROR가 있는 줄만 선택함.

```bash
awk '/ERROR/{print}' app.log
```

19. ERROR 줄 번호 표시

원본 로그의 ERROR 줄 번호와 내용을 표시함.

```bash
awk '/ERROR/{print NR ": " $0}' app.log
```

20. ERROR/WARN 동시 검색

ERROR 또는 WARN이 있는 줄을 찾음.

```bash
awk '/ERROR|WARN/{print NR ": " $0}' app.log
```

21. 특정 줄 범위 출력

10~30줄만 선택하여 출력함.

```bash
awk 'NR>=10 && NR<=30 {print NR ": " $0}' member.jsp
```

22. 첫 번째 필드 출력

공백 기준 첫 번째 필드만 출력함.

```bash
awk '{print $1}' file.txt
```

23. 여러 필드 출력

첫 번째와 세 번째 필드를 선택함.

```bash
awk '{print $1,$3}' file.txt
```

24. 구분자 지정

:을 구분자로 사용하여 Linux 사용자명만 추출함.

```bash
awk -F: '{print $1}' /etc/passwd
```

25. ERROR 개수 계산

ERROR가 발생한 총 건수를 계산함.

```bash
awk '/ERROR/{n++} END{print n+0}' app.log
```

26. ERROR/WARN 통계

두 종류의 로그 건수를 한 번에 계산함.

```bash
awk '/ERROR/{e++} /WARN/{w++} END{print "ERROR=",e+0,"WARN=",w+0}' app.log
```

27. 숫자 합계

첫 번째 필드의 숫자를 모두 합산함.

```bash
awk '{sum+=$1} END{print "합계=",sum}' data.txt
```

28. 숫자 평균

첫 번째 필드의 평균을 계산함.

```bash
awk '{sum+=$1} END{if(NR) print "평균=",sum/NR}' data.txt
```

29. 특정 값 이상 선택

세 번째 필드 값이 80보다 큰 행만 선택함.

```bash
awk '$3>80 {print}' data.txt
```

30. JSP form 영역 추출

```bash
<form>부터 </form>까지의 코드 영역만 추출함.
awk '/<form/,/<\/form>/{print NR ": " $0}' member.jsp
```

5. awk 추가 실무 응용

ERROR와 WARN을 구분하여 출력:

```bash
awk '
/ERROR/ {print "[ERROR]", NR, $0}
/WARN/  {print "[WARN ]", NR, $0}
' app.log
```

Linux 사용자명과 UID 출력:

```bash
awk -F: '{
    print "USER=" $1, "UID=" $3
}' /etc/passwd
```

UID 1000 이상 계정 확인:

```bash
awk -F: '$3 >= 1000 {
    print $1, $3
}' /etc/passwd
```

디스크 사용률 80% 이상 경고:

```bash
df -P | awk '
NR > 1 {
    use=$5
    gsub("%","",use)
    if (use >= 80)
        print "WARNING:", $1, use "%"
}'
```

JSP script 영역 추출:

```bash
awk '/<script/,/<\/script>/ {
    print NR ": " $0
}' member.jsp
```

6. sed + awk + tee 병합 예제

application.properties에서 주석과 빈 줄을 제거한 뒤 실제 설정에 번호를 붙여 출력:

```bash
sed -e '/^[[:space:]]*#/d' -e '/^[[:space:]]*$/d' application.properties |
awk '{print NR ": " $0}'
```

ERROR/WARN을 추출하고 번호를 붙여 보고서 파일에도 저장:

```bash
sed -n '/ERROR\|WARN/p' app.log |
awk '{print NR ": " $0}' |
tee error-warn-report.txt
```

안전한 설정 변경의 권장 순서:

```bash
grep -n 'server.port' application.properties
cp application.properties application.properties.bak
sed -i 's/^server\.port=.*/server.port=8081/' application.properties
grep -n 'server.port' application.properties | tee change-result.txt
```

7. 실무 안전 원칙

- sed -i로 운영 파일을 수정하기 전에 sed 명령에서 -i를 빼고 변경 결과를 먼저 확인하는 습관이 좋음.
- 실제 수정 시 sed -i.bak 또는 cp 명령으로 원본을 백업하는 것이 안전함.
- find와 sed를 결합하면 여러 파일이 동시에 바뀌므로 grep/find로 대상 파일을 먼저 확인해야 함.
- awk는 원본 파일을 직접 수정하는 도구라기보다 검색, 필드 가공, 조건 분석, 집계에 활용하는 것이 이해하기 쉬움.
- 로그 및 설정 분석 결과를 남길 때 tee를 조합하면 화면 확인과 보고서 저장을 동시에 할 수 있음.

부록 · 리눅스 명령어

Linux awk · sed · grep · tee 실전 교재

편집기 없이 JSP · conf · properties · YAML · Java · Kotlin 검색/치환/삭제/자동화

1. 핵심 개념

쉘에서 설정파일과 소스코드를 다룰 때 find는 대상 파일 탐색, grep은 문자열·구문 검색, awk는 조건·범위·필드 가공, sed는 치환·삽입·삭제, tee는 화면 출력과 결과 저장에 사용함.

권장 흐름:

```bash
find → grep → 백업 → awk → sed → grep 검증 → tee 보고서 → 필요 시 cron 자동화
```

2. 핵심 명령 비교

명령	역할	대표 기능	예
find	파일 탐색	확장자·경로·조건별 파일 검색	find . -name "*.jsp"
grep	문자열 검색	줄 번호, 재귀, 정규식, 주변 문맥	grep -Rni "ERROR" .
awk	조건/범위 가공	줄 선택, 필드 처리, 집계	awk '/ERROR/{print NR,$0}' app.log
sed	텍스트 편집	치환, 삭제, 삽입, 일괄 수정	sed -i.bak 's/8080/8081/g' file
tee	출력+저장	화면 확인과 파일 기록 동시 수행	grep ERROR app.log | tee errors.txt

3. 편집기 없이 사용하는 실전 명령·예제 35개 이상

1. 문자열과 줄 번호 검색

ERROR가 들어간 줄과 줄 번호를 표시함.

```bash
grep -n "ERROR" app.log
```

2. 대소문자 무시 검색

ERROR, Error, error 등을 모두 찾음.

```bash
grep -ni "error" app.log
```

3. 하위 폴더 전체 검색

현재 경로 아래 모든 파일을 재귀 검색함.

```bash
grep -Rni "ERROR" .
```

4. JSP 파일만 검색

JSP 안에서 session 사용 위치를 찾음.

```bash
grep -Rni --include="*.jsp" "session" .
```

5. Java/Kotlin 동시 검색

Java와 Kotlin 소스의 TODO를 찾음.

```bash
grep -Rni --include="*.java" --include="*.kt" "TODO" .
```

6. 에러 앞뒤 문맥 확인

ERROR 앞뒤 3줄을 함께 표시함.

```bash
grep -n -C 3 "ERROR" app.log
```

7. ERROR/WARN 동시 검색

확장 정규식으로 두 패턴을 동시에 검색함.

```bash
grep -En "ERROR|WARN" app.log
```

8. DEBUG 제외 출력

DEBUG가 없는 줄만 출력함.

```bash
grep -v "DEBUG" app.log
```

9. awk로 ERROR와 줄 번호 출력

NR을 이용해 실제 원본 줄 번호와 내용을 출력함.

```bash
awk '/ERROR/{print NR ": " $0}' app.log
```

10. awk로 ERROR/WARN 선택

둘 중 하나가 있는 줄을 선택함.

```bash
awk '/ERROR|WARN/{print NR ": " $0}' app.log
```

11. 특정 줄 범위 출력

20~40줄만 편집기 없이 확인함.

```bash
awk 'NR>=20 && NR<=40 {print NR ": " $0}' test.jsp
```

12. JSP form 구간 출력

form 시작부터 종료 태그까지 출력함.

```bash
awk '/<form/,/<\/form>/{print NR ": " $0}' member.jsp
```

13. JSP script 구간 출력

JavaScript 구간을 추출함.

```bash
awk '/<script/,/<\/script>/{print NR ": " $0}' member.jsp
```

14. JSP 스크립틀릿 구간 출력

JSP <% ... %> 영역을 추출함.

```bash
awk '/<%/,/%>/{print NR ": " $0}' member.jsp
```

15. ERROR 개수 집계

로그 전체의 ERROR 건수를 계산함.

```bash
awk '/ERROR/{n++} END{print "ERROR:",n}' app.log
```

16. 기본 문자열 치환

파일은 바꾸지 않고 변경 결과만 화면에 미리 보여줌.

```bash
sed 's/8080/8081/g' application.properties
```

17. 실제 파일 치환

application.properties를 직접 수정함.

```bash
sed -i 's/8080/8081/g' application.properties
```

18. 백업 후 안전 치환

.bak 원본을 남긴 뒤 수정함.

```bash
sed -i.bak 's/8080/8081/g' application.properties
```

19. 특정 줄 삭제

10번째 줄을 삭제하며 백업을 남김.

```bash
sed -i.bak '10d' member.jsp
```

20. 줄 범위 삭제

10~20번째 줄을 삭제함.

```bash
sed -i.bak '10,20d' member.jsp
```

21. DEBUG 포함 줄 삭제

DEBUG 문자열이 있는 줄만 선택 삭제함.

```bash
sed -i.bak '/DEBUG/d' application.properties
```

22. 빈 줄 제거

빈 줄을 제거한 결과를 화면에 출력함.

```bash
sed '/^[[:space:]]*$/d' application.properties
```

23. 주석 줄 제거

#으로 시작하는 주석을 제외하고 출력함.

```bash
sed '/^[[:space:]]*#/d' application.properties
```

24. 특정 줄 뒤에 설정 삽입

server.port 줄 다음에 주소 설정을 삽입해 미리 확인함.

```bash
sed '/server.port/a server.address=0.0.0.0' application.properties
```

25. 특정 줄 앞에 주석 삽입

해당 설정 바로 앞에 설명 주석을 넣음.

```bash
sed '/server.port/i # Server Setting' application.properties
```

26. 파일 끝에 설정 추가

기존 파일 끝에 설정을 추가함.

```bash
echo "server.port=8080" | tee -a application.properties
```

27. root 설정파일에 추가

관리자 권한이 필요한 파일에 내용을 추가함.

```bash
echo "TEST=1" | sudo tee -a /etc/test.conf
```

28. 검색 결과 보고서 저장

화면에서 확인하면서 errors.txt에도 저장함.

```bash
grep -n "ERROR" app.log | tee errors.txt
```

29. awk 분석 결과 저장

```bash
awk 결과를 화면과 파일에 동시에 기록함.
awk '/ERROR/{print NR,$0}' app.log | tee errors.txt
```

30. JSP 전체에서 session 찾기

모든 JSP 파일의 파일명·줄 번호·구문을 출력함.

```bash
find . -name "*.jsp" -exec grep -nH "session" {} +
```

31. conf 전체에서 listen 찾기

conf 설정의 listen 위치를 찾음.

```bash
find /etc -name "*.conf" -exec grep -nH "listen" {} +
```

32. JSP 여러 파일 일괄 치환

모든 JSP에서 OLD를 NEW로 바꾸고 각 파일 백업을 남김.

```bash
find . -name "*.jsp" -exec sed -i.bak "s/OLD/NEW/g" {} +
```

33. properties에서 password 위치 검색

비밀번호 관련 설정의 위치를 찾음.

```bash
find . -name "*.properties" -exec grep -nH "password" {} +
```

34. TODO 보고서 만들기

미완료 코드 위치를 보고서로 저장함.

```bash
grep -Rni "TODO" src/ | tee todo.txt
```

35. IP 주소 사용 위치 검색

프로젝트에서 특정 IP가 사용된 위치를 찾음.

```bash
grep -Rni "192\.168\.0\.50" . | tee ip.txt
```

36. 여러 설정 확장자에서 localhost 검색

Spring 설정파일의 localhost 사용 위치를 검색함.

```bash
grep -Rni --include="*.yml" --include="*.yaml" --include="*.properties" "localhost" .
```

37. 수정 전후 검증

검색→백업→수정→재검색의 안전한 기본 패턴임.

```bash
grep -n '8080' application.properties | tee before.txt
sed -i.bak 's/8080/8081/g' application.properties
grep -n '8081' application.properties | tee after.txt
```

4. 자동화 Shell Script 예제 20개

1. JSP 에러 문자열 자동 검색

```bash
#!/bin/bash
DIR="/var/lib/tomcat/webapps"
REPORT="/tmp/jsp-error.txt"
grep -Rni --include="*.jsp" "error" "$DIR" | tee "$REPORT"
```

2. Java/Kotlin TODO 자동 검색

```bash
#!/bin/bash
grep -Rni --include="*.java" --include="*.kt" "TODO" src/ | tee todo-report.txt
```

3. Spring Boot 포트 자동 변경

```bash
#!/bin/bash
FILE="src/main/resources/application.properties"
cp "$FILE" "$FILE.bak"
sed -i 's/^server\.port=.*/server.port=8081/' "$FILE"
grep -n '^server\.port=' "$FILE" | tee port-result.txt
```

4. MariaDB 주소 자동 변경

```bash
#!/bin/bash
FILE="application.properties"
cp "$FILE" "$FILE.bak"
sed -i 's/192\.168\.0\.10/SERVER_IP/g' "$FILE"
grep -n "jdbc:mariadb" "$FILE" | tee database-result.txt
```

5. 설정파일 localhost 자동 검색

```bash
#!/bin/bash
grep -Rni --include="*.properties" --include="*.yml" --include="*.yaml" --include="*.conf" "lo
calhost" . | tee localhost-report.txt
```

6. ERROR/WARN 자동 분석

```bash
#!/bin/bash
LOG="app.log"
awk '
/ERROR/ {print "[ERROR]", NR, $0}
/WARN/  {print "[WARN ]", NR, $0}
' "$LOG" | tee error-warn-report.txt
```

7. ERROR/WARN 건수 자동 통계

```bash
#!/bin/bash
awk '
/ERROR/ {error++}
/WARN/  {warn++}
END {
  print "ERROR =", error+0
  print "WARN  =", warn+0
}
' app.log | tee log-statistics.txt
```

8. JSP DEBUG 코드 자동 검색

```bash
#!/bin/bash
find . -type f -name "*.jsp" -exec grep -niH "DEBUG" {} + | tee jsp-debug.txt
```

9. JSP DEBUG 줄 백업 후 삭제

```bash
#!/bin/bash
find . -type f -name "*.jsp" -exec sed -i.bak '/DEBUG/d' {} +
echo "DEBUG 제거 완료" | tee debug-delete.log
```

10. properties 자동 백업

```bash
#!/bin/bash
DATE=$(date +%Y%m%d-%H%M%S)
BACKUP="backup-$DATE"
mkdir -p "$BACKUP"
find . -type f -name "*.properties" -exec cp --parents {} "$BACKUP" \;
echo "Backup: $BACKUP" | tee backup.log
```

11. 설정파일 전체 자동 백업

```bash
#!/bin/bash
BACKUP="config-$(date +%Y%m%d-%H%M%S)"
mkdir -p "$BACKUP"
find . -type f \( -name "*.properties" -o -name "*.yml" -o -name "*.yaml" -o -name "*.conf" \)
-exec cp --parents {} "$BACKUP" \;
echo "$BACKUP" | tee backup-result.txt
```

12. localhost 일괄 변경

```bash
#!/bin/bash
find . -type f \( -name "*.properties" -o -name "*.yml" -o -name "*.yaml" \)   -exec sed -i.ba
k 's/localhost/SERVER_IP/g' {} +
grep -Rni "SERVER_IP" . | tee replace-result.txt
```

13. JSP form 코드 자동 추출

```bash
#!/bin/bash
FILE="member.jsp"
awk '/<form/,/<\/form>/{print NR ": " $0}' "$FILE" | tee form-code.txt
```

14. JSP 스크립틀릿 자동 추출

```bash
#!/bin/bash
FILE="member.jsp"
awk '/<%/,/%>/{print NR ": " $0}' "$FILE" | tee jsp-java-code.txt
```

15. Spring Boot 설정 자동 검사

```bash
#!/bin/bash
FILE="application.properties"
{
 echo "===== PORT ====="
 grep -n '^server\.port' "$FILE"
 echo "===== DATABASE ====="
 grep -n 'datasource' "$FILE"
 echo "===== JPA ====="
 grep -n 'jpa' "$FILE"
} | tee config-report.txt
```

16. MariaDB 설정 위치 자동 추출

```bash
#!/bin/bash
grep -Rni --include="*.properties" --include="*.yml" --include="*.yaml" "mariadb" . | tee mari
adb-config.txt
```

17. 비밀번호 설정 파일 위치만 검사

```bash
#!/bin/bash
grep -Rnil --include="*.properties" --include="*.yml" --include="*.yaml" "password" . | tee pa
ssword-files.txt
```

18. Java/Kotlin/JSP 통합 코드 검색

```bash
#!/bin/bash
SEARCH="$1"
if [ -z "$SEARCH" ]; then
  echo "사용법: $0 검색어"
  exit 1
fi
grep -Rni --include="*.java" --include="*.kt" --include="*.jsp" "$SEARCH" . | tee search-result.txt
```

19. 서버 로그 일일 자동 분석

```bash
#!/bin/bash
LOG="/home/devuser/app/app.log"
DATE=$(date +%Y%m%d)
REPORT="/home/devuser/report-$DATE.txt"
{
 echo "===== SERVER REPORT ====="
 date
 echo "===== ERROR ====="
 grep -ni "ERROR" "$LOG" | tail -50
 echo "===== WARN ====="
 grep -ni "WARN" "$LOG" | tail -50
 echo "===== COUNT ====="
 awk '/ERROR/{e++} /WARN/{w++} END{print "ERROR:",e+0; print "WARN:",w+0}' "$LOG"
} | tee "$REPORT"
```

20. 프로젝트 통합 자동 검사

```bash
#!/bin/bash
PROJECT="/home/devuser/app"
REPORT="$PROJECT/project-report.txt"
cd "$PROJECT" || exit 1
{
 echo "================================"
 echo "PROJECT CHECK"
 date
 echo "===== JAVA PROCESS ====="
 pgrep -af java || echo "Java process 없음"
 echo "===== PORT 8080 ====="
 ss -tulpn | grep 8080 || echo "8080 port 없음"
 echo "===== ERROR ====="
 grep -ni "ERROR" app.log | tail -20
 echo "===== WARN ====="
 grep -ni "WARN" app.log | tail -20
 echo "===== ERROR/WARN COUNT ====="
 awk '/ERROR/{e++} /WARN/{w++} END{print "ERROR:",e+0; print "WARN:",w+0}' app.log
 echo "===== SERVER PORT ====="
 grep -Rni --include="*.properties" --include="*.yml" "server.port" .
 echo "===== DATABASE ====="
 grep -Rni --include="*.properties" --include="*.yml" "datasource" .
 echo "===== TODO ====="
 grep -Rni --include="*.java" --include="*.kt" --include="*.jsp" "TODO" .
 echo "===== DISK ====="
 df -h
 echo "===== MEMORY ====="
 free -h
} | tee -a "$REPORT"
```

5. cron 자동 실행 예

스크립트에 실행 권한을 부여한 뒤 crontab -e에서 등록함.

```bash
chmod +x project-check.sh
0 9 * * * /home/devuser/project-check.sh
0 23 * * * /home/devuser/log-report.sh
```

첫 번째 예는 매일 오전 9시, 두 번째 예는 매일 오후 11시에 실행함.

6. 실무 안전 원칙

- sed -i 또는 find와 sed를 결합한 일괄 수정 전에 grep으로 변경 대상을 먼저 확인함.
- 가능하면 sed -i.bak 또는 별도 백업 디렉터리를 사용하여 원본을 보존함.
- 자동 삭제는 검색 보고서를 먼저 만든 뒤 사람이 확인하고 적용하는 2단계 방식이 안전함.
- password, API key 등 비밀정보는 tee 보고서에 실제 값이 남지 않도록 파일명·위치만 검사하는 것이 좋음.
- 운영 서버의 Spring Boot 프로세스 자동 재시작은 단순 cron보다 systemd의 Restart 정책을 우선 검토함.

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.

부록 · 리눅스 명령어

Linux sed · awk · tee 자동화 실전 100제

JSP · Java · Kotlin · Spring Boot · MariaDB · Linux 서버 관리

이 문서에서 고친 곳

- 여러 파일을 awk 로 훑으며 줄 번호를 NR 로 찍던 28곳을 FNR 로 고쳤습니다. NR 은 파일을 넘어 이어지는 번호라 두 번째 파일부터 줄 번호가 틀립니다(실측: 두 번째 파일 첫 줄이 3 으로 나옴).

학습 개요

sed는 치환·삭제·삽입, awk는 조건 검색·필드 처리·계산·통계, tee는 화면 출력과 파일 저장을 동시에 수행함. 이 문서는 세 명령을 find, grep, cron 등과 결합하여 실제 운영 자동화에 적용하는 100개 예제를 정리함.

안전 원칙: 검색 → 대상 확인 → 백업 → 수정 → 재검증 → tee로 결과 저장 순서를 권장함.

1. sed 중심 파일·설정 변경 자동화

1. Spring Boot 포트 변경 + 결과 저장

```bash
FILE="application.properties"
sed -i.bak 's/^server\.port=.*/server.port=8081/' "$FILE"
awk '/^server\.port=/{print}' "$FILE" | tee port-change.log
```

2. localhost를 서버 IP로 변경

```bash
sed -i.bak 's/localhost/SERVER_IP/g' application.properties
awk '/192\.168\.0\.50/{print}' application.properties | tee change.log
```

3. MariaDB IP 변경

```bash
sed -i.bak 's/192\.168\.0\.10/SERVER_IP/g' application.properties
awk '/jdbc:mariadb/{print}' application.properties | tee mariadb-change.log
```

4. JSP 회원가입 문구 변경

```bash
sed -i.bak 's/회원가입/회원등록/g' member.jsp
awk '/회원등록/{print NR ":" $0}' member.jsp | tee member-change.log
```

5. DEBUG 줄 자동 삭제

```bash
sed -i.bak '/DEBUG/d' application.properties
awk '/DEBUG/{print NR ":" $0}' application.properties | tee debug-check.log
```

6. 빈 줄 자동 제거

```bash
sed '/^[[:space:]]*$/d' application.properties | tee application.clean.properties
```

7. 주석 줄 제거 결과 저장

```bash
sed '/^[[:space:]]*#/d' application.properties | tee no-comment.properties
```

8. 10번째 줄 삭제

```bash
cp member.jsp member.jsp.bak
sed -i '10d' member.jsp
awk 'NR>=8&&NR<=12{print NR ":" $0}' member.jsp | tee line-check.txt
```

9. 10~20번째 줄 삭제

```bash
cp member.jsp member.jsp.bak
sed -i '10,20d' member.jsp
awk 'NR>=8&&NR<=15{print NR ":" $0}' member.jsp | tee range-check.txt
```

10. server.port 뒤에 server.address 삽입

```bash
sed -i.bak '/^server\.port=/a server.address=0.0.0.0' application.properties
awk '/server\.port|server\.address/{print}' application.properties | tee server-config.txt
```

11. datasource 앞에 설명 주석 삽입

```bash
sed -i.bak '/^spring\.datasource/i # Database Configuration' application.properties
sed -n '/Database Configuration/,+3p' application.properties | tee db-section.txt
```

12. JSP title 변경

```bash
sed -i.bak 's#<title>.*</title>#<title>WizData</title>#' index.jsp
awk '/<title>/{print NR ":" $0}' index.jsp | tee title.log
```

13. JSP DEBUG 블록 삭제

```bash
sed -i.bak '/<!-- DEBUG/,/DEBUG -->/d' index.jsp
awk '/DEBUG/{print NR ":" $0}' index.jsp | tee debug-remain.log
```

14. 여러 공백 정리

```bash
sed 's/[[:space:]][[:space:]]*/ /g' file.txt | tee clean.txt
```

15. 앞 공백 제거

```bash
sed 's/^[[:space:]]*//' file.txt | tee left-trim.txt
```

16. 뒤 공백 제거

```bash
sed 's/[[:space:]]*$//' file.txt | tee right-trim.txt
```

17. 앞뒤 공백 동시 제거

```bash
sed 's/^[[:space:]]*//;s/[[:space:]]*$//' file.txt | tee trim.txt
```

18. datasource 설정만 추출

```bash
sed -n '/spring\.datasource/p' application.properties | tee db-config.txt
```

19. JSP 100~150줄 추출

```bash
sed -n '100,150p' member.jsp | awk '{print NR+99 ":" $0}' | tee jsp-code.txt
```

20. properties 파일 포트 일괄 변경

```bash
find . -name "*.properties" -exec sed -i.bak 's/^server\.port=.*/server.port=8081/' {} +
find . -name "*.properties" -exec awk '/server\.port/{print FILENAME ":" FNR ":" $0}' {} + | tee ports.txt
```

2. awk 중심 검색·분석·통계 자동화

21. ERROR 자동 검색

```bash
awk '/ERROR/{print NR ":" $0}' app.log | tee error.log
```

22. WARN 자동 검색

```bash
awk '/WARN/{print NR ":" $0}' app.log | tee warn.log
```

23. ERROR + WARN 동시 검색

```bash
awk '/ERROR|WARN/{print NR ":" $0}' app.log | tee warning.log
```

24. ERROR 개수 자동 계산

```bash
awk '/ERROR/{n++} END{print "ERROR =",n+0}' app.log | tee error-count.txt
```

25. ERROR/WARN 통계

```bash
awk '/ERROR/{e++} /WARN/{w++} END{print "ERROR =",e+0; print "WARN =",w+0}' app.log | tee statistics.txt
```

26. JSP form 부분 추출

```bash
awk '/<form/,/<\/form>/{print NR ":" $0}' member.jsp | tee form.txt
```

27. JSP script 부분 추출

```bash
awk '/<script/,/<\/script>/{print NR ":" $0}' member.jsp | tee script.txt
```

28. JSP 스크립틀릿 부분 추출

```bash
awk '/<%/,/%>/{print NR ":" $0}' member.jsp | tee jsp-java.txt
```

29. 50~100줄 추출

```bash
awk 'NR>=50&&NR<=100{print NR ":" $0}' file.txt | tee range.txt
```

30. Linux 사용자명 추출

```bash
awk -F: '{print $1}' /etc/passwd | tee users.txt
```

31. 사용자명 + UID 추출

```bash
awk -F: '{print $1,$3}' /etc/passwd | tee user-uid.txt
```

32. UID 1000 이상 계정 찾기

```bash
awk -F: '$3>=1000{print $1,$3}' /etc/passwd | tee normal-users.txt
```

33. CSV 첫 번째 열 추출

```bash
awk -F, '{print $1}' data.csv | tee col1.txt
```

34. CSV 첫 번째·세 번째 열 추출

```bash
awk -F, '{print $1,$3}' data.csv | tee selected-columns.txt
```

35. 숫자 합계 계산

```bash
awk '{sum+=$1} END{print "TOTAL =",sum}' data.txt | tee total.txt
```

36. 숫자 평균 계산

```bash
awk '{sum+=$1} END{if(NR) print "AVERAGE =",sum/NR}' data.txt | tee average.txt
```

37. 세 번째 필드 80 이상 추출

```bash
awk '$3>=80{print}' data.txt | tee high.txt
```

38. 여러 로그에서 ERROR 위치 출력

```bash
awk '/ERROR/{print FILENAME ":" FNR ":" $0}' *.log | tee all-error.txt
```

39. Java TODO 자동 검색

```bash
find src -name "*.java" -exec awk '/TODO/{print FILENAME ":" FNR ":" $0}' {} + | tee java-todo.txt
```

40. Kotlin TODO 자동 검색

```bash
find src -name "*.kt" -exec awk '/TODO/{print FILENAME ":" FNR ":" $0}' {} + | tee kotlin-todo.txt
```

3. tee 중심 로그·보고서 자동화

41. 디스크 상태 보고서

```bash
df -h | tee disk.txt
```

42. 날짜를 기존 로그에 추가

```bash
date | tee -a server.log
```

43. 메모리 상태 보고서

```bash
free -h | tee memory.txt
```

44. 프로세스 전체 저장

```bash
ps aux | tee process.txt
```

45. Java 프로세스 저장

```bash
pgrep -af java | tee java-process.txt
```

46. 열린 포트 상태 저장

```bash
ss -tulpn | tee ports.txt
```

47. 8080 포트만 저장

```bash
ss -tulpn | awk '/:8080/{print}' | tee port8080.txt
```

48. 서버 종합 기본 보고서

```bash
{ date; uptime; free -h; df -h; } | tee server-report.txt
```

49. 한 결과를 두 파일에 동시 저장

```bash
echo "SERVER OK" | tee server1.log server2.log
```

50. application.properties 끝에 설정 추가

```bash
echo "server.port=8080" | tee -a application.properties
```

51. root 설정파일 작성

```bash
echo "TEST=1" | sudo tee /etc/test.conf
```

52. Spring Boot 실행 로그 저장

```bash
java -jar app.jar 2>&1 | tee app.log
```

53. Spring Boot 실행 로그 누적

```bash
java -jar app.jar 2>&1 | tee -a app.log
```

54. HTTP 응답 저장

```bash
curl -s http://localhost:8080 | tee response.html
```

55. 환경변수 목록 저장

```bash
env | tee env.txt
```

4. JSP · Java · Kotlin 코드 분석 자동화

56. JSP request.getParameter 검색

```bash
find . -name "*.jsp" -exec awk '/request\.getParameter/{print FILENAME ":" FNR ":" $0}' {} + | tee request-check.txt
```

57. JSP session 사용 검색

```bash
find . -name "*.jsp" -exec awk '/session\./{print FILENAME ":" FNR ":" $0}' {} + | tee session-check.txt
```

58. JSP System.out.println 검색

```bash
find . -name "*.jsp" -exec awk '/System\.out\.println/{print FILENAME ":" FNR ":" $0}' {} + | tee jsp-println.txt
```

59. JSP TODO 검색

```bash
find . -name "*.jsp" -exec awk '/TODO/{print FILENAME ":" FNR ":" $0}' {} + | tee jsp-todo.txt
```

60. Java System.out.println 검색

```bash
find src -name "*.java" -exec awk '/System\.out\.println/{print FILENAME ":" FNR ":" $0}' {} + | tee java-print.txt
```

61. Kotlin println 검색

```bash
find src -name "*.kt" -exec awk '/println\(/{print FILENAME ":" FNR ":" $0}' {} + | tee kotlin-print.txt
```

62. Java public 메서드 검색

```bash
find src -name "*.java" -exec awk '/public .*[[:space:]]*[A-Za-z0-9_]+\(/{print FILENAME ":" FNR ":" $0}' {} + | tee java-methods.txt
```

63. Kotlin fun 검색

```bash
find src -name "*.kt" -exec awk '/^[[:space:]]*fun /{print FILENAME ":" FNR ":" $0}' {} + | tee kotlin-functions.txt
```

64. Repository 검색

```bash
find src -type f \( -name "*.java" -o -name "*.kt" \) -exec awk '/Repository/{print FILENAME ":" FNR ":" $0}' {} + | tee repositories.txt
```

65. @Entity 검색

```bash
find src -type f \( -name "*.java" -o -name "*.kt" \) -exec awk '/@Entity/{print FILENAME ":" FNR ":" $0}' {} + | tee entities.txt
```

66. DTO 관련 코드 검색

```bash
find src -type f \( -name "*.java" -o -name "*.kt" \) -exec awk '/Dto|DTO/{print FILENAME ":" FNR ":" $0}' {} + | tee dtos.txt
```

67. Controller 검색

```bash
find src -type f \( -name "*.java" -o -name "*.kt" \) -exec awk '/@RestController|@Controller/{print FILENAME ":" FNR ":" $0}' {} + | tee controllers.txt
```

68. Service 검색

```bash
find src -type f \( -name "*.java" -o -name "*.kt" \) -exec awk '/@Service/{print FILENAME ":" FNR ":" $0}' {} + | tee services.txt
```

69. JSP href 검색

```bash
find . -name "*.jsp" -exec awk '/href=/{print FILENAME ":" FNR ":" $0}' {} + | tee href.txt
```

70. JSP form action 검색

```bash
find . -name "*.jsp" -exec awk '/action=/{print FILENAME ":" FNR ":" $0}' {} + | tee actions.txt
```

5. Spring Boot 설정·로그 자동화

71. server.port 검색

```bash
awk '/server\.port/{print NR ":" $0}' application.properties | tee port.txt
```

72. datasource.url 검색

```bash
awk '/datasource\.url/{print NR ":" $0}' application.properties | tee database.txt
```

73. JPA 설정 추출

```bash
awk '/spring\.jpa/{print NR ":" $0}' application.properties | tee jpa.txt
```

74. Spring profile 추출

```bash
awk '/spring\.profiles/{print NR ":" $0}' application.properties | tee profile.txt
```

75. 포트 변경 후 자동 검증

```bash
sed -i.bak 's/^server\.port=.*/server.port=8081/' application.properties
awk '/server\.port/{print}' application.properties | tee port-result.txt
```

76. DB IP 변경 후 자동 검증

```bash
sed -i.bak 's/192\.168\.0\.10/SERVER_IP/g' application.properties
awk '/datasource\.url/{print}' application.properties | tee database-result.txt
```

77. ddl-auto를 update로 변경

```bash
sed -i.bak 's/^spring\.jpa\.hibernate\.ddl-auto=.*/spring.jpa.hibernate.ddl-auto=update/'
application.properties
awk '/ddl-auto/{print}' application.properties | tee ddl.txt
```

78. show-sql을 true로 변경

```bash
sed -i.bak 's/^spring\.jpa\.show-sql=.*/spring.jpa.show-sql=true/' application.properties
awk '/show-sql/{print}' application.properties | tee show-sql.txt
```

79. root logging level을 INFO로 변경

```bash
sed -i.bak 's/^logging\.level\.root=.*/logging.level.root=INFO/' application.properties
awk '/logging\.level\.root/{print}' application.properties | tee logging.txt
```

80. application.yml 포트 변경

```bash
sed -i.bak '/^[[:space:]]*port:/s/[0-9][0-9]*/8081/' application.yml
awk '/^[[:space:]]*port:/{print NR ":" $0}' application.yml | tee yml-port.txt
```

81. Spring Boot ERROR 통계

```bash
awk '/ERROR/{e++} END{print "ERROR:",e+0}' app.log | tee spring-error.txt
```

82. 최근 ERROR 20건 저장

```bash
awk '/ERROR/{print NR ":" $0}' app.log | tail -20 | tee last-error.txt
```

83. 최근 WARN 20건 저장

```bash
awk '/WARN/{print NR ":" $0}' app.log | tail -20 | tee last-warn.txt
```

84. Exception 자동 검색

```bash
awk '/Exception/{print NR ":" $0}' app.log | tee exceptions.txt
```

85. Spring Boot 로그 종합 통계

```bash
awk '/ERROR/{e++} /WARN/{w++} /Exception/{x++} END{print "ERROR =",e+0; print "WARN =",w+
0; print "Exception =",x+0}' app.log | tee boot-stat.txt
```

6. Linux 서버 상태 점검 자동화

86. 디스크 80% 이상 경고

```bash
df -P | awk 'NR>1{u=$5; gsub("%","",u); if(u>=80) print "WARNING:",$1,u"%",$6}' | tee disk
-warning.txt
```

87. 메모리 사용량 보고서

```bash
free -m | awk 'NR==2{print "Total:",$2,"MB"; print "Used:",$3,"MB"; print "Free:",$4,"MB"}' | tee memory-report.txt
```

88. CPU load 기록

```bash
uptime | awk '{print}' | tee load.txt
```

89. Java 프로세스 자동 점검

```bash
pgrep -af java | awk '{print "JAVA:",$0}' | tee java-status.txt
```

90. 8080 포트 점검

```bash
ss -tulpn | awk '/:8080/{print}' | tee 8080-status.txt
```

91. 3306 MariaDB 포트 점검

```bash
ss -tulpn | awk '/:3306/{print}' | tee mariadb-port.txt
```

92. SSH 로그인 기록 분석

```bash
last | awk '{print $1,$3,$4,$5,$6}' | tee login-report.txt
```

93. root 로그인 기록만 추출

```bash
last | awk '$1=="root"{print}' | tee root-login.txt
```

94. 홈 디렉터리 용량 보고서

```bash
du -sh /home/* 2>/dev/null | sort -h | tee home-size.txt
```

95. 서버 기본 상태 통합 저장

```bash
{ date; hostname; uptime; df -h; free -h; } | tee server-status.txt
```

7. 종합 Shell Script 자동화

96. Spring Boot 자동 상태 보고서

```bash
#!/bin/bash
LOG="app.log"
{
 echo "===== SERVER REPORT ====="
 date
 echo "[JAVA]"
 pgrep -af java
 echo "[PORT]"
 ss -tulpn | awk '/:8080/'
 echo "[ERROR]"
 awk '/ERROR/{print NR ":" $0}' "$LOG" | tail -10
 echo "[ERROR COUNT]"
 awk '/ERROR/{e++} END{print e+0}' "$LOG"
} | tee server-report.txt
```

97. 설정파일 자동 백업 + 변경

```bash
#!/bin/bash
FILE="application.properties"
cp "$FILE" "$FILE.$(date +%Y%m%d-%H%M%S).bak"
sed -i 's/^server\.port=.*/server.port=8081/' "$FILE"
awk '/server\.port/{print "NEW CONFIG:",$0}' "$FILE" | tee configuration-change.txt
```

98. JSP 자동 코드 품질 점검

```bash
#!/bin/bash
find . -name "*.jsp" -exec awk '
/System\.out\.println/{print "[PRINT]",FILENAME,FNR,$0}
/DEBUG/{print "[DEBUG]",FILENAME,FNR,$0}
/TODO/{print "[TODO]",FILENAME,FNR,$0}
/request\.getParameter/{print "[PARAMETER]",FILENAME,FNR,$0}
' {} + | tee jsp-check.txt
```

99. Spring Boot 프로젝트 자동 검사

```bash
#!/bin/bash
{
 echo "===== PORT ====="
 find src/main/resources -type f \( -name "*.properties" -o -name "*.yml" \)    -exec awk '/se
rver\.port|port:/{print FILENAME ":" FNR ":" $0}' {} +
 echo "===== DATABASE ====="
 find src/main/resources -type f    -exec awk '/datasource|mariadb|jdbc/{print FILENAME ":" FN
R ":" $0}' {} +
 echo "===== CONTROLLER ====="
 find src -type f \( -name "*.java" -o -name "*.kt" \)    -exec awk '/@RestController|@Control
ler/{print FILENAME ":" FNR ":" $0}' {} +
 echo "===== TODO ====="
 find src -type f \( -name "*.java" -o -name "*.kt" -o -name "*.jsp" \)    -exec awk '/TODO/{p
rint FILENAME ":" FNR ":" $0}' {} +
} | tee project-check.txt
```

100. sed + awk + tee 서버 종합 자동화

```bash
#!/bin/bash
PROJECT="/home/devuser/app"
LOG="$PROJECT/app.log"
CONFIG="$PROJECT/application.properties"
REPORT="$PROJECT/daily-report-$(date +%Y%m%d).txt"
cd "$PROJECT" || exit 1
{
 echo "========================================"
 echo "      SERVER AUTOMATION REPORT"
 echo "========================================"
 date
 echo "===== HOST ====="
 hostname
 echo "===== JAVA PROCESS ====="
 pgrep -af java || echo "Java process 없음"
 echo "===== PORT 8080 ====="
 ss -tulpn | awk '/:8080/{print}'
 echo "===== PORT 3306 ====="
 ss -tulpn | awk '/:3306/{print}'
 echo "===== SPRING CONFIG ====="
 awk '/server\.port/{print "[PORT]",$0} /datasource\.url/{print "[DATABASE]",$0} /spring\.jpa/
{print "[JPA]",$0}' "$CONFIG"
 echo "===== LOG STATISTICS ====="
 awk '/ERROR/{e++} /WARN/{w++} /Exception/{x++} END{print "ERROR =",e+0; print "WARN =",w+0; p
rint "EXCEPTION =",x+0}' "$LOG"
 echo "===== RECENT ERRORS ====="
 awk '/ERROR/{print NR ":" $0}' "$LOG" | tail -20
 echo "===== DISK ====="
 df -P | awk 'NR>1{u=$5; gsub("%","",u); print $1,u"%",$6; if(u>=80) print ">>> WARNING: DISK
usage high"}'
 echo "===== MEMORY ====="
 free -m | awk 'NR==2{print "Total =",$2,"MB"; print "Used =",$3,"MB"; print "Free =",$4,"M
B"}'
 echo "===== TODO CODE ====="
 find src -type f \( -name "*.java" -o -name "*.kt" -o -name "*.jsp" \)    -exec awk '/TODO/{p
rint FILENAME ":" FNR ":" $0}' {} +
 echo "========================================"
 echo "             COMPLETE"
 echo "========================================"
} | tee "$REPORT"
```

8. cron 자동 실행 예제

스크립트 실행 권한을 부여한 뒤 crontab에 등록하면 정기 자동화를 구성할 수 있음.

```bash
chmod +x server-report.sh
crontab -e

# 매일 오전 9시 실행
0 9 * * * /home/devuser/app/server-report.sh

# 매일 오후 11시 로그 분석
0 23 * * * /home/devuser/app/log-report.sh
```

9. 실무 안전 체크리스트

- sed -i 실행 전에는 가능하면 -i를 빼고 화면에서 예상 결과를 먼저 확인함.
- 운영 설정파일은 sed -i.bak 또는 날짜가 포함된 별도 백업 파일을 만든 뒤 수정함.
- find + sed 일괄 변경은 대상 파일 목록을 먼저 출력하여 범위를 확인함.
- 비밀번호, API 키, 토큰 등 민감한 설정값을 tee 보고서에 그대로 기록하지 않도록 주의함.
- 자동 삭제와 자동 변경은 테스트 환경에서 검증한 후 운영 서버에 적용함.
- cron 실행 시 상대경로보다 절대경로를 사용하는 것이 안전함.

부록 · 리눅스 명령어

Linux tee 명령어 상세 설명과 자동화 실습

tee 예제 10개 · Shell Script/cron 자동화 예제 5개

1. tee 명령어란?

tee는 표준 입력(stdin)으로 전달된 내용을 화면(표준 출력)에 표시하면서 동시에 하나 이상의 파일에 저장하는 명령어임. 명령 결과를 눈으로 확인하면서 로그 파일에도 남길 때 매우 유용함.

기본 형식: 명령어 | tee 파일명

추가 기록: 명령어 | tee -a 파일명

핵심 흐름: 명령어 → 파이프(|) → tee → 화면 출력 + 파일 저장

2. tee 핵심 옵션

형식/옵션	의미	예
tee file	화면 출력과 동시에 파일에 저장(기존 내용 덮어쓰기 가능)	df -h | tee disk.txt
tee -a file	기존 파일 끝에 내용을 추가(append)	df -h | tee -a disk.txt
tee file1 file2	동일한 내용을 여러 파일에 동시에 저장	df -h | tee a.txt b.txt
2>&1 | tee	표준 오류(stderr)까지 합쳐서 저장	java -jar app.jar 2>&1 | tee app.log
| sudo tee	root 권한이 필요한 파일에 파이프 결과 기록	echo TEST | sudo tee /etc/test.conf

3. tee 명령어 실전 예제 10개

1. 문자열 출력과 저장

화면에 Hello Linux를 표시하면서 test.txt에 동일한 내용을 저장함.

```bash
echo "Hello Linux" | tee test.txt
```

2. 기존 파일에 이어쓰기

-a는 append의 의미로 기존 내용을 보존하고 마지막에 내용을 추가함.

```bash
echo "추가 내용" | tee -a test.txt
```

3. 시스템 정보 저장

커널 및 시스템 정보를 화면에서 확인하면서 system.txt에 저장함.

```bash
uname -a | tee system.txt
```

4. 디스크 사용량 저장

디스크 전체 사용량을 사람이 읽기 쉬운 단위로 표시하고 파일에도 저장함.

```bash
df -h | tee disk.txt
```

5. 메모리 상태 저장

현재 메모리와 스왑 사용 상태를 memory.txt에 기록함.

```bash
free -h | tee memory.txt
```

6. 프로세스 목록 저장

실행 중인 프로세스 목록을 화면과 파일에 동시에 출력함.

```bash
ps aux | tee process.txt
```

7. 여러 파일에 동시 저장

동일한 결과를 disk1.txt와 disk2.txt 두 파일에 동시에 기록함.

```bash
df -h | tee disk1.txt disk2.txt
```

8. 관리자 권한 파일 작성

리다이렉션 권한 문제를 피하면서 /etc 아래의 파일에 root 권한으로 기록할 때 유용함.

```bash
echo "TEST" | sudo tee /etc/test.conf
```

9. Spring Boot 로그 저장

정상 출력(stdout)과 오류 출력(stderr)을 합쳐 화면에 표시하고 app.log에도 저장함.

```bash
java -jar app.jar 2>&1 | tee app.log
```

10. Spring Boot 로그 누적

기존 app.log를 지우지 않고 새로운 실행 로그를 계속 추가함.

```bash
java -jar app.jar 2>&1 | tee -a app.log
```

4. 표준 출력과 표준 오류 이해

Linux에서 일반 출력은 stdout(파일 디스크립터 1), 오류 메시지는 stderr(파일 디스크립터 2)로 구분됨. 따라서 프로그램의 오류까지 tee에 전달하려면 2>&1을 사용함.

```bash
java -jar app.jar 2>&1 | tee -a app.log
```

의미: stderr(2)를 stdout(1)으로 합침 → 파이프로 tee에 전달 → 화면 표시와 app.log 기록을 동시에 수행함.

5. 자동화 실행 예제 5개

자동화 1. 매일 서버 상태 기록

파일명: server-check.sh

```bash
#!/bin/bash

LOG="/home/devuser/server-status.log"

echo "============================" | tee -a "$LOG"
date | tee -a "$LOG"

echo "[DISK]" | tee -a "$LOG"
df -h | tee -a "$LOG"

echo "[MEMORY]" | tee -a "$LOG"
free -h | tee -a "$LOG"

echo "[LOAD]" | tee -a "$LOG"
uptime | tee -a "$LOG"
```

실행 권한 부여:

```bash
chmod +x server-check.sh
```

매일 오전 9시 실행:

```text
0 9 * * * /home/devuser/server-check.sh
```

자동화 2. Spring Boot 자동 실행 + 로그 저장

파일명: start-app.sh

```bash
#!/bin/bash

APP="/home/devuser/app/app.jar"
LOG="/home/devuser/app/app.log"

echo "===== Spring Boot START =====" | tee -a "$LOG"
date | tee -a "$LOG"

java -jar "$APP" 2>&1 | tee -a "$LOG"
```

실행 권한 부여:

```bash
chmod +x start-app.sh
```

터미널 종료 후에도 실행:

```bash
nohup ./start-app.sh >/dev/null 2>&1 &
```

자동화 3. MariaDB 상태 자동 점검

파일명: mariadb-check.sh

```bash
#!/bin/bash

LOG="/home/devuser/mariadb-check.log"

echo "========================" | tee -a "$LOG"
date | tee -a "$LOG"

systemctl status mariadb --no-pager | tee -a "$LOG"
ss -tulpn | grep 3306 | tee -a "$LOG"
```

실행 권한 부여:

```bash
chmod +x mariadb-check.sh
```

10분마다 실행:

```text
*/10 * * * * /home/devuser/mariadb-check.sh
```

자동화 4. 디스크 사용량 자동 기록

파일명: disk-check.sh

```bash
#!/bin/bash

LOG="/home/devuser/disk.log"

echo "====================" | tee -a "$LOG"
date | tee -a "$LOG"

df -h | tee -a "$LOG"

echo "[Large directories]" | tee -a "$LOG"
du -sh /home/* 2>/dev/null | sort -h | tee -a "$LOG"
```

실행 권한 부여:

```bash
chmod +x disk-check.sh
```

매일 밤 11시 실행:

```text
0 23 * * * /home/devuser/disk-check.sh
```

자동화 5. Spring Boot 상태 확인 후 재시작

파일명: monitor.sh

```bash
#!/bin/bash

URL="http://localhost:8080"
APP="/home/devuser/app/app.jar"
LOG="/home/devuser/app/monitor.log"

echo "==============================" | tee -a "$LOG"
date | tee -a "$LOG"

if curl -fsS "$URL" > /dev/null
then
    echo "Spring Boot 정상 작동" | tee -a "$LOG"
else
    echo "Spring Boot 응답 없음" | tee -a "$LOG"
    echo "Spring Boot 재시작" | tee -a "$LOG"
    nohup java -jar "$APP" >> "$LOG" 2>&1 &
fi
```

실행 권한 부여:

```bash
chmod +x monitor.sh
```

5분마다 실행:

```text
*/5 * * * * /home/devuser/app/monitor.sh
```

6. cron 기본 사용법

```bash
crontab -e 명령으로 현재 사용자의 예약 작업을 편집함. cron 형식은 '분 시 일 월 요일 실행명령' 순서임.
```

설정	의미	예
0 9 * * *	매일 오전 9시	0 9 * * * /home/devuser/server-check.sh
*/5 * * * *	5분마다	*/5 * * * * /home/devuser/app/monitor.sh
*/10 * * * *	10분마다	*/10 * * * * /home/devuser/mariadb-check.sh
0 23 * * *	매일 오후 11시	0 23 * * * /home/devuser/disk-check.sh

7. 실무 주의사항

- tee는 기본적으로 파일을 덮어쓸 수 있으므로 운영 로그를 누적하려면 tee -a를 사용하는 것이 안전함.
- cron에서 실행할 스크립트에는 chmod +x 파일명으로 실행 권한을 부여해야 함.
- cron 환경은 로그인 셸과 환경변수가 다를 수 있으므로 운영 자동화에서는 가능한 한 실행 파일의 절대경로와 로그 경로를 명확히 지정하는 것이 좋음.
- Spring Boot 운영 서버에서는 단순 nohup/cron 감시보다 systemd 서비스 등록과 Restart 정책을 사용하는 방식이 일반적으로 더 안정적임.

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.